University

Project Showcase

A research-grade OS written
spec-first, test-driven

Phases 0–18 complete • 399/399 tests

GrahaOS

A Capability-First, AI-Native Operating System

Built from scratch • x86_64 • Modular monolith → hybrid microkernel

Team

Divyesh Bine	100522733073	CSE Sem 8
Farhan Ahmed	100522733079	CSE Sem 8
Abraar Ahmed	100522733075	CSE Sem 8

Why a New Operating System?

The assumptions underneath Unix are 50 years old.

What current OSes were designed for

- Multi-user timesharing on expensive hardware (1970s).
- Users, uid/gid, root — ambient authority.
- Fork + exec, signals, blocking read()/write().
- Files-as-bytes: no semantic meaning at OS level.
- Drivers welded into the kernel for speed.

What modern workloads need

- AI agents are a new class of user — must be sandboxable.
- Authority should be explicit, revocable, audit-traceable.
- Batched async I/O — syscalls cost too much per op.
- The OS should understand file semantics (type hashes, embeddings).
- Safe speculation: 'try this and roll back if wrong'.

GrahaOS rebuilds the kernel around capabilities, manifests, and speculation — native, not bolted-on.

The Seven Design Principles

Every spec file must uphold at least two of these and violate none.

1. **Capabilities are the sole authority**
   No ambient auth. No users. Identity = tokens held.

2. **GCP is the only native public interface**
   Every syscall manifest-typed. POSIX is optional userspace.

3. **Foundation before migration**
   Lay new primitives, then move old callers behind shims.

4. **Failure is auditable, not silent**
   Panics dump structured frames. Violations hit the audit log.

5. **Everything is testable by hand**
   Every spec has manual_verification with exact commands.

6. **Drivers drift outward**
   Kernel keeps VMM/sched/interrupts. Net/FS/block migrate out.

7. **Speculation is a first-class AI primitive**
   Versioned FS + snapshots + txn channels = safe AI exploration.

Architecture Overview

Modular monolith today, hybrid microkernel by Phase 23.

USERSPACE

gash / grahai

shell + AI agent

ktest

TAP test harness

libc / libtap / libgcp

thin libraries

Future drivers (P21-23)

net / AHCI / block

IPC

KERNEL

cap/ + io/ + ipc/ + mm/

Tokens • Pledges • Channels • VMOs • Streams

fs/ — VFS + GrahaFS + SimHash

AI-metadata native, async_read hooks

arch/x86_64/ — scheduler + VMM + PMM

TASK_STATE_CHAN_WAIT, HHDM mapping, per-CPU

drivers (in-kernel today, leaving P21-23)

AHCI / e1000 / lapic / keyboard / fb

Kernel boundary moves outward each phase. Today it owns scheduler + MM + drivers. Phase 21-23: drivers out.

Capabilities as the Sole Authority

No users. No uid/gid. No root. Identity = bundle of tokens held.

cap_token_t — 64 bits

Generation 32 bits	Object Index 24 bits	Flags 8

Passed by value in registers. Never heap-allocated.

Resolve path (hot)

- Atomic-acquire load generation → compare → reject if stale.
- Check calling pid in audience_set (CAP_FLAG_PUBLIC bypass).
- Bitmask AND check on rights (READ / WRITE / DERIVE / ...).
- No locks. ~20–30 cycles typical.

Key properties

Derivable

Narrow rights / audience into child caps; tree tracked.

Revocable

Single gen++ invalidates all outstanding tokens (O(1)).

Audience-scoped

Audience set is per-capability; privacy-preserving filter.

Kind-dispatched

CHANNEL, VMO, STREAM, FILE, PROC — each has a deactivator.

Pledges — The Second Authority Axis

'What WILL this process ever do again?' — monotonically narrowing.

Concept

- Capability answers: CAN do this now?
- Pledge answers: WILL ever do this again?
- Narrow-only: drop PLEDGE_FS_WRITE once → permanent.
- Reset only on SYS_SPAWN (new child, fresh subset).
- Every sensitive syscall = one AND + one compare.

12 Classes (16-bit mask)

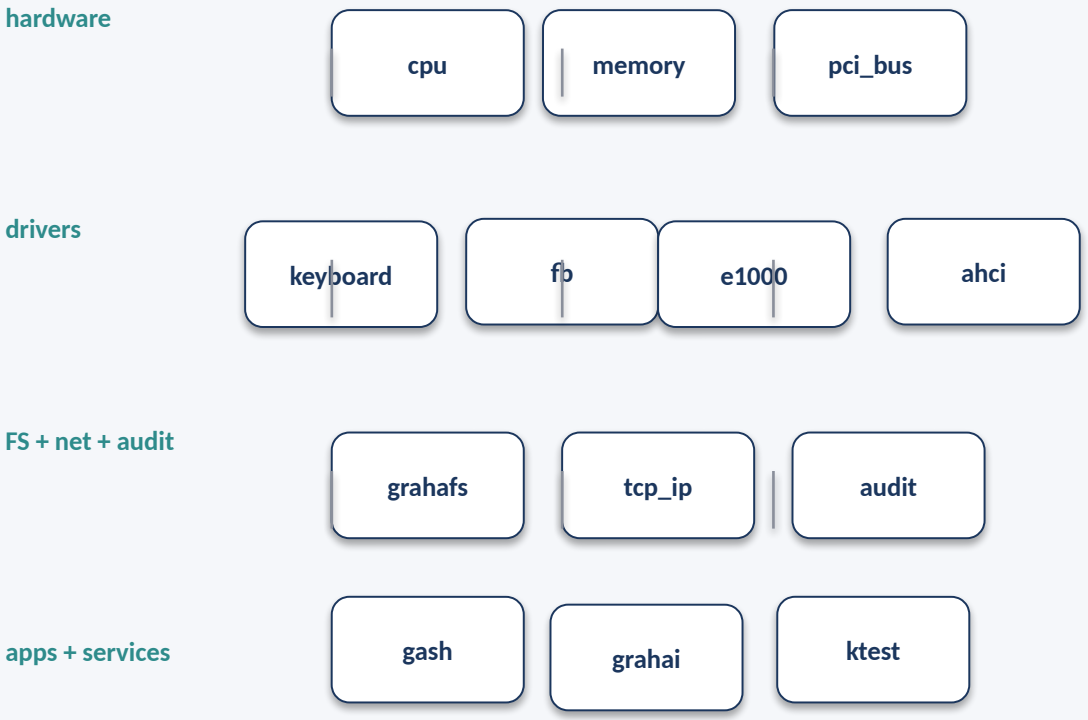
FS_READ files read	**FS_WRITE** files write	**NET_CLIENT** outbound net
NET_SERVER listen/bind	**SPAWN** create processes	**IPC_SEND** channel send
IPC_RECV channel recv	**SYS_QUERY** read kernel state	**SYS_CONTROL** reboot / kill
AI_CALL AI agent	**COMPUTE** CPU + memory	**TIME** sleep / clock

```
pledge_allows(task, CLASS)  =  (mask.raw & (1u << CLASS)) != 0   — header-only inline, zero overhead.
```

CAN — Capability Activation Network

The sole registry of system components. Drivers as graph nodes.

- Every driver / service / app registered as a CAN cap.
- Dependency edges: e.g., tcp_ip depends on e1000 depends on pci_bus.
- Activate cascades forward; deactivate cascades backward.
- Each cap has on_activate / on_deactivate callbacks — physically drives hardware (keyboard IRQ on/off, fb mmio on/off).
- Audit log entries on every state flip.
- Phase 16 completed the hardware wiring (keyboard, fb, e1000, ahci).



→ activate cascades up; deactivate cascades down.

GCP — The Only Native Public Interface

/etc/gcp.json is the single source of truth for every syscall and message.

What GCP enforces

- Every syscall has a manifest entry in JSON.
- Every channel message carries a 64-bit type_hash (FNV-1a).
- Kernel rejects mistyped sends at entry (−EPROTOTYPE).
- Every stream op has a generated op_schema row.
- AI agents + humans read the same JSON file.
- POSIX is optional libposix in userspace — never in kernel.

/etc/gcp.json (excerpt)

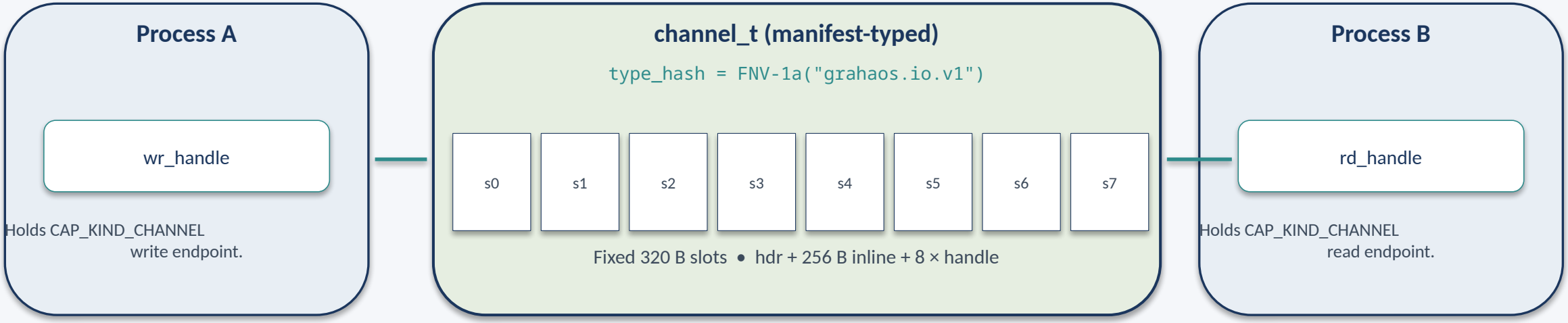
```
"grahaos.io.v1": {
  "ops": {
    "READ_VMO": { "op": 1,
      "pledge": "FS_READ",
      "dispatcher": "dispatch_read_vmo" },
    "WRITE_VMO":  { "op": 2, … },
    "SENDMSG":    { "op": 3, … }
  }
}
```

gen_manifest.py → single JSON → kernel op_schema + userspace OP_* constants.

Kernel and userspace can never drift out of sync — the ABI is generated.

Channels — Typed Bidirectional IPC

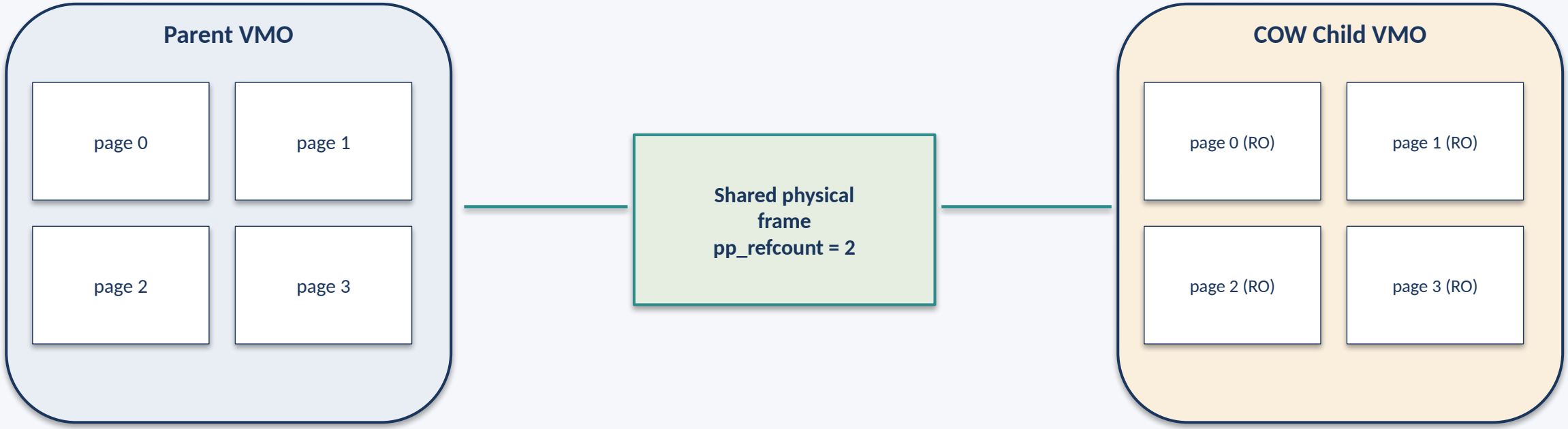
Replace ambient pipes. Atomic handle transfer. Manifest-anchored.



- Two CAP_KIND_CHANNEL cap_objects per channel — read endpoint + write endpoint.
- chan_send with mismatched type_hash → −EPROTOTYPE + audit entry.
- Atomic handle transfer: handles moved out of sender's table into the slot, then into receiver.
- TASK_STATE_CHAN_WAIT + per-tick deadline scan give timeouts without calibration.

Virtual Memory Objects (VMOs)

Named, refcounted physical frames. Copy-on-write — the speculation substrate.

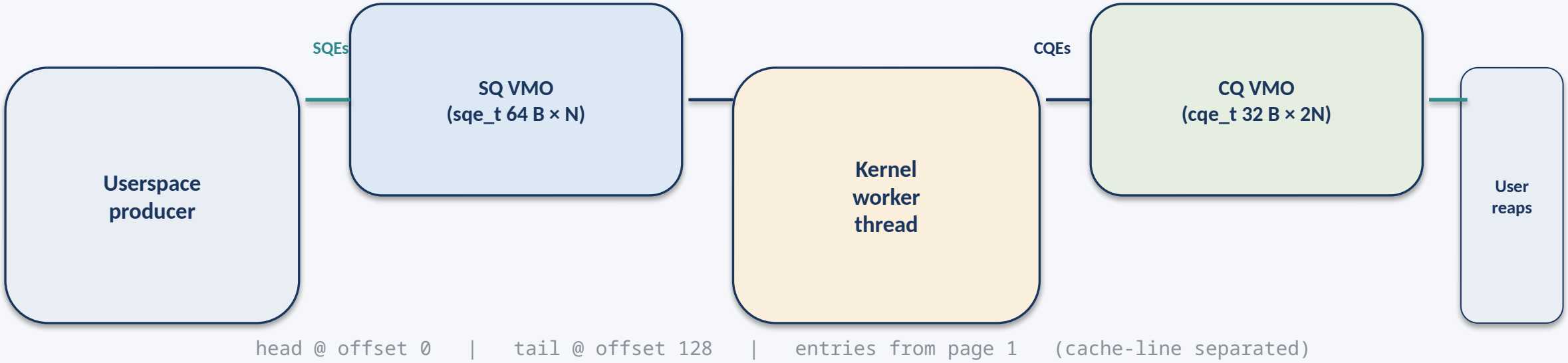


Why VMOs matter

- Zero-copy shared memory between kernel and userspace (HHDM direct-map).
- COW isolation: write to a shared frame triggers private-copy via page-fault handler.
- Per-physical-page refcount (1 byte parallel to PMM bitmap).
- Substrate for Phase 25 speculative transactions — "try and roll back" for free.

Submission Streams — Async I/O

io_uring-style batching, GCP-native. Every op manifest-validated.



- One SYS_STREAM_SUBMIT can hand off thousands of ops — amortizes user/kernel crossings.
- Every SQE type-validated against the stream's manifest at kernel entry.
- Per-op pledge check; bad ops get inline CQE with −EPROTOTYPE or −EPLEDGE.
- ACQUIRE / RELEASE on head/tail — io_uring memory discipline, documented in stream.h.
- Optional notify channel: non-blocking event loop without polling.

GrahaFS — Filesystem with AI Semantics

AI metadata and similarity clustering are first-class.

On-disk structure

Superblock

magic, sizes, free bitmap

Inode table

256-byte inodes (not 128)

Extended meta

tags, importance, ai_embedding[128]

Data blocks

4 KiB each; 48 KiB file cap → Phase 19 fixes

SimHash Clustering (Phase 11)

- 64-bit fingerprint per file (FNV-1a over 3-char shingles).
- Sequential-leader clustering: Hamming distance ≤ 10 → same cluster.
- Cluster IDs stored in inode's ai_reserved[].
- Background indexer task scans root every 3 s.
- /illu/ directory for deterministic files (no auto-cluster).

```
simhash("hello world this is a test") = 0x7F2A4B…  |  popcount(a ^ b) = Hamming distance
```

OS-level similarity — no external ML dependency.

AI as a First-Class Citizen

Not bolted on. Baked in at the filesystem and speculation layers.

grahai

Embedded AI agent process. Calls Gemini via in-kernel TCP/IP stack (Phase 9e).

PLEDGE_AI_CALL + audit log entry per invoke.

ai_metadata on every file

tags[], importance score, ai_embedding[128], last_ai_touched.

Queryable via SYS_SET_AI_METADATA / SYS_GET_AI_METADATA.

SimHash similarity

Find near-duplicate files in Hamming distance — no external embedding service.

OS surfaces cluster membership as metadata.

Speculative execution (Phase 25)

Transactions on COW VMO snapshots. AI tries a change; OS rolls back cleanly if wrong.

This is the differentiator — every other OS nails 'safety' but not safe exploration.

The Spec-Driven Workflow

Every phase starts as a YAML spec. No speculative coding.

1 — YAML spec

specs/phase-NN.yml
actor workflows
manual verification

2 — Implement

new source files
struct + syscall
manifests

3 — TAP tests

make test
gate_tests_passing
stability iters

4 — Manual VP

shell commands
exact expected
outputs match

5 — Audit

problems_faced.md
audit_log.json
per-issue root cause

6 — Gate next

only start N+1 after
N exit-criteria green

Why this works

Every phase has a reviewable YAML contract before a line of C is written. Every audit entry in problems_faced.md becomes an invariant for future phases — we know why the code is the way it is.

Testing & Verification Floor

TAP 1.4. Every spec has a manual_verification section. make test exits zero or fails.

399/399	31	15/15	26s	9
assertions passing	TAP test files	stability iterations clean	full suite wall time	issues logged in phase 18

Structure per phase

- unit_tests — isolated property checks.
- integration_tests — full-system workflows.
- fault_injection_tests — deliberate failures.
- stress_tests — soak at scale.
- gate: yes vs gate: no per entry.

Notable tests

- sentinel_meta proves the harness catches failures.
- cantest_v2 (37) / captest_v2 (42) — capability lifecycle.
- chantest (24) / vmotest (18) — Phase 17 IPC substrate.
- streamtest (22) — Phase 18 async I/O correctness.
- canstress 100× cap cycle — no stuck IRQs.

Phases Completed (0 → 18)

Boot to Phase 18 async I/O — 18 substantial spec-driven milestones.

Foundation

P0–7	Bootloader, GDT/IDT, PMM/VMM, sched
P8	CAN registry + drivers (kb/fb/lapic)
P9	E1000 + Mongoose TCP/IP + HTTPS + DNS + Gemini
P10	Per-process FDs + pipes + redirects
P11	SimHash + sequential-leader clustering

Recent — spec-driven cycle

P12	Test harness + TAP 1.4 floor
P13	Structured logging + panic/oops dump
P14	Slab + kheap + per-CPU (NUMA-ready)
P15a	Capability Objects v2 (tokens, gen, audience)
P15b	Pledges + persistent audit log
P16	CAN callbacks → physical hardware control
P17	Channels + VMOs + handle-passing IPC
P18	**Submission Streams (async I/O) ← current**

Roadmap 19 → 28

Toward an MVP: versioned FS, userspace drivers, speculation, Wasmtime, TUI.

P19	**GrahaFS v2** versioned segments + full data journaling; async_read consumed	P20	**Per-CPU scheduler** work-stealing runqueues; resource limits
P21	**Userspace driver framework** e1000 migration out of kernel	P22	**In-tree TCP/IP** replace 19 k-line Mongoose; ARP/IPv4/UDP/TCP-Reno/DHCP
P23	**AHCI userspace** interrupt-driven block I/O; completes drivers-outward	P24	**COW snapshot subsystem** whole-process snapshots on VMO infra
P25	**Speculative execution** transactions on snapshots — the AI-exploration primitive	P26	**Wasmtime userspace** GCP as WIT; Rust user programs
P27	**TUI framework** virtual consoles + widgets	P28	**MVP** TUI-first shell + AI polish + 1-hour stress bench

Phase 25 is the thesis: snapshots + COW VMO + transactional channels = safe AI speculation.

Key Novel Contributions

What's actually new — vs. Unix / Linux / microkernels / io_uring.

No users, no root

Identity = bundle of cap_tokens. uid/gid/root never existed.

Dual authority (cap + pledge)

Caps say CAN now. Pledges say WILL ever again. Orthogonal.

GCP as sole public interface

Every syscall and message validates against JSON manifest at entry.

Manifest-typed channels

Ambient-authority pipes dead. Every message type-hash-anchored.

Speculation as OS primitive

COW VMOs + snapshots + transactional channels (coming P24–25).

Generated ABI — kernel ↔ user single-source

gen_manifest.py emits both sides. Drift impossible.

AI metadata in the filesystem

tags, importance, embeddings, SimHash clusters — native.

Everything auditable

Caps, pledges, stream rejections, VMO faults — all in /var/audit.

Conclusion

An AI-native OS built the slow way — spec first, test floor high.

Where we are

Phases 0–18 complete. 399/399 tests. Async I/O shipped. Kernel ready to shed drivers.

What's next

Phase 19 versioned FS, then drivers-outward (P21–23), then speculation (P24–25).

Why it matters

Every other OS treats AI as an app. We treat AI as a user class with its own safety needs.

Thank you

Questions? — github.com/B-Divyesh/GrahaOS